



WWW

World Wide Web



World Wide Web (WWW)

La **World Wide Web (WWW)** es un sistema global de **recursos interconectados** accesibles a través de Internet.

Actualmente la Web no solo permite acceder a documentos, sino que constituye una **plataforma para el desarrollo de aplicaciones distribuidas**.



¿Cómo funciona?

Cuando un usuario accede a una página Web, el navegador sigue una serie de pasos clave para recuperar y mostrar el contenido:

1. **Solicitud del recurso:** Se ingresa una **URL** (Uniform Resource Locator) en la barra de direcciones o se hace clic en un hipervínculo. Por ejemplo, <https://www.ubp.edu.ar>.
2. **Resolución de nombre de dominio (DNS):** La parte del dominio en la URL (ubp.edu.ar) se traduce en una dirección IP mediante el **Sistema de Nombres de Dominio (DNS)**.

Una vez obtenida la **dirección IP** del servidor mediante DNS, el navegador establece una conexión de red con el servidor y envía una solicitud utilizando el protocolo HTTP.

¿Cómo funciona? (Cont.)

3. **Establecimiento de conexión con el servidor:** Se inicia una conexión utilizando **TCP** o **UDP (en HTTP/3)** para establecer comunicación con el servidor Web.
4. **Petición HTTP al servidor:** El navegador envía una **solicitud HTTP** al servidor para obtener el recurso solicitado.
HTTP es el protocolo encargado de la transferencia de información en la Web.
5. **Recepción y renderización de la página Web:** El servidor responde con los **datos del recurso** (página HTML, imágenes, scripts, etc.).
El **navegador interpreta y muestra el contenido** al usuario.

Cliente Web

Un **cliente Web**, también llamado **navegador**, es un programa que permite a los usuarios **acceder a páginas Web y otros recursos disponibles en Internet**.

Cuando un usuario solicita una página Web, el navegador:

- ▶ **Solicita el documento HTML** al servidor mediante una **petición HTTP**.
- ▶ **Interpreta el código HTML** para construir la estructura de la página.
- ▶ **Solicita recursos adicionales** como **CSS, JavaScript e imágenes**.
- ▶ **Renderiza la página** en la interfaz del usuario y permite la **navegación mediante hipervínculos**.

Servidor Web

Un **servidor Web** es un programa que **recibe y responde solicitudes HTTP** enviadas por los **clientes Web** (navegadores).

Sus principales funciones son:

- ▶ **Procesar peticiones HTTP** provenientes de los clientes.
- ▶ **Servir recursos Web** como **HTML**, imágenes, hojas de estilo y scripts.
- ▶ **Gestionar múltiples sitios Web** mediante **virtual hosting**.
- ▶ **Administrar aspectos de seguridad**, como **autenticación y control de acceso**.

Procesamiento en el servidor

En este modelo, la **lógica principal de la aplicación se ejecuta en el servidor**, que procesa las solicitudes del usuario y genera la página que será enviada al navegador.

Flujo de interacción:

- ▶ El usuario realiza una acción en el navegador (por ejemplo, enviar un formulario o hacer clic en un enlace).
- ▶ El navegador envía una solicitud HTTP al servidor Web.
- ▶ El servidor **procesa la solicitud**, **accede a la base de datos si es necesario** y genera una **respuesta en HTML**.
- ▶ El navegador recibe el documento HTML y **renderiza la página**.

Procesamiento en el cliente

En este modelo, parte de la **lógica de la aplicación se ejecuta en el navegador** mediante **JavaScript**, permitiendo actualizar la interfaz sin recargar toda la página.

Flujo de interacción:

- ▶ El servidor envía un **documento HTML inicial** al navegador.
- ▶ El navegador ejecuta **JavaScript** que controla el comportamiento de la aplicación.
- ▶ A medida que el usuario interactúa, el navegador solicita **datos adicionales al servidor**.
- ▶ La interfaz se **actualiza dinámicamente** sin necesidad de recargar la página completa.

URI: Identificador de Recursos

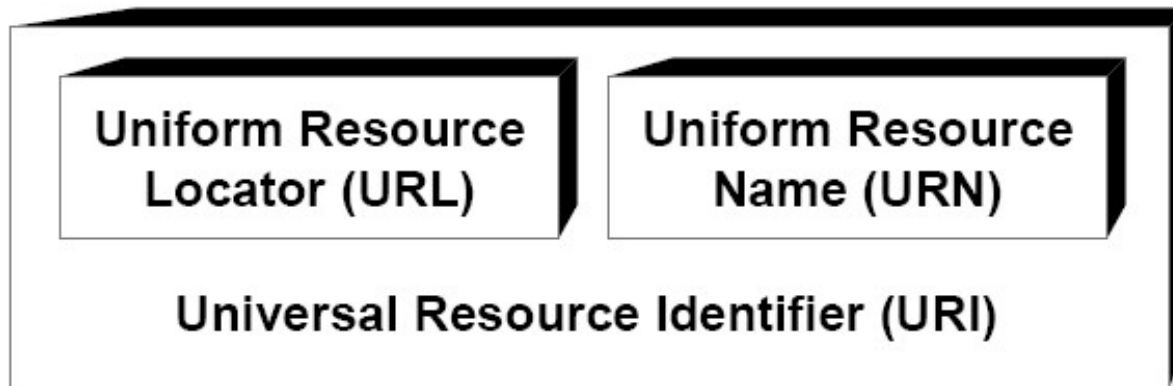
Un **URI (Uniform Resource Identifier)** es un estándar utilizado para **identificar recursos de forma única en Internet**.

Un recurso puede ser:

- ▶ una página Web
- ▶ una imagen
- ▶ un archivo
- ▶ un recurso en una API

El URI proporciona una **forma estructurada de identificar estos recursos dentro de la Web y otros sistemas distribuidos**.

Tipos de URI



- ▶ **URL (Uniform Resource Locator):** Es el tipo de URI más común.
Además de identificar un recurso, **indica su ubicación en Internet**. Ejemplo: <https://www.ubp.edu.ar/>
- ▶ **URN (Uniform Resource Name):** Identifica un recurso de **manera única**, pero **sin especificar su ubicación**.
Ejemplo: <urn:isbn:0451450523>

Importancia de los URI

Los URI son fundamentales para el funcionamiento de la Web porque:

- ▶ permiten la **identificación única de recursos**
- ▶ son utilizados por **HTTP para acceder a recursos**
- ▶ son la base de **APIs REST**
- ▶ facilitan la **interoperabilidad en sistemas distribuidos**



Componentes de un URL

<https://guest:secret@www.ubp.edu.ar:443/carreras/index.jsp?nombre=INFORMATICA#plan>

Descomposición	
https://	Protocolo
guest:secret@	Credenciales (usuario y contraseña) [opcional]
www.ubp.edu.ar	Host / Dominio del servidor
:443	Puerto de comunicación
/carreras/	Ruta dentro del servidor
index.jsp	Recurso solicitado
?nombre=INFORMATICA	Parámetros (Query)
#plan	Fragmento o ancla

Componentes de un URL (Cont.)

- ▶ **Protocolo:** define cómo se realizará la comunicación (HTTP, HTTPS, FTP, etc.).
- ▶ **Host / Dominio:** identifica el servidor donde se encuentra el recurso.
- ▶ **Puerto:** canal de comunicación del servicio (HTTPS usa normalmente 443).
- ▶ **Ruta y recurso:** ubicación del archivo o recurso dentro del servidor.
- ▶ **Parámetros (query):** datos enviados al servidor en formato clave=valor.
- ▶ **Fragmento:** referencia a una sección específica dentro del recurso.

Codificación de URL (URL encoding)

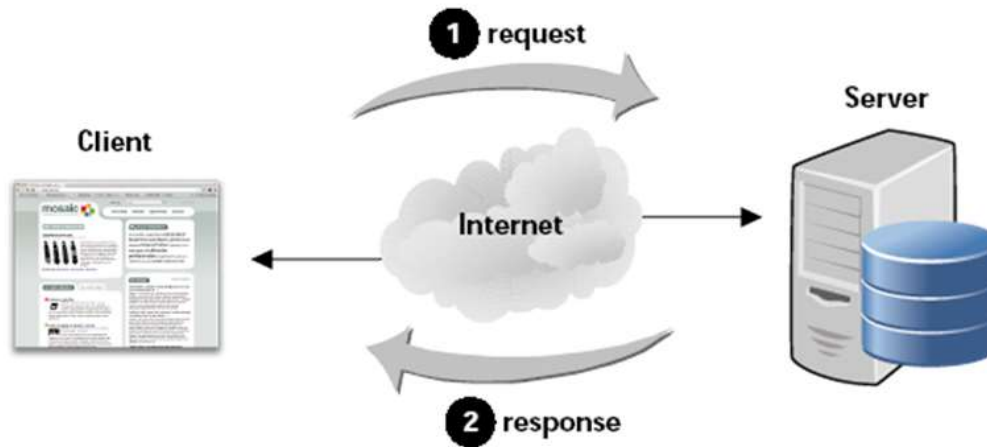
Es un mecanismo que permite **representar caracteres especiales dentro de una dirección Web** de forma segura y compatible con los estándares de Internet.

Se utiliza para:

- ▶ **Garantizar la correcta transmisión de caracteres** dentro de las direcciones Web.
- ▶ **Representar caracteres especiales o reservados** que podrían interferir con la estructura de la URL.
- ▶ **Codificar caracteres no imprimibles** mediante una representación segura.

En este proceso, los caracteres se reemplazan por un **código hexadecimal precedido por el símbolo %**.

HTTP



HTTP (HyperText Transfer Protocol) es el **protocolo de comunicación** utilizado para la **transferencia de información** que permite la comunicación entre clientes y servidores Web.

HTTP es un protocolo sin estado (stateless).

HTTP: Evolución

- ▶ **HTTP/0.9 (1991):** Protocolo muy básico, solo permitía la transferencia de documentos HTML sin encabezados.
- ▶ **HTTP/1.0 (1996):** Introdujo encabezados de respuesta, permitiendo definir el tipo de contenido.
- ▶ **HTTP/1.1 (1997):** Implementó conexiones persistentes, compresión de datos y pipelining para mejorar el rendimiento.
- ▶ **HTTP/2 (2015):** Introdujo la multiplexación, permitiendo múltiples solicitudes simultáneas en una sola conexión.
- ▶ **HTTP/3 (QUIC) (2022):** Basado en UDP, elimina problemas de congestión, reduce la latencia en redes inestables y mejora la seguridad con TLS 1.3.

HTTP: SSL/TLS y seguridad en la comunicación

SSL (Secure Sockets Layer) y su evolución TLS (Transport Layer Security) son **protocolos criptográficos** que aseguran la **transmisión segura** de datos en la Web.

Importancia de TLS:

- ▶ **Cifrado:** Protege la información contra ataques de interceptación.
- ▶ **Integridad:** Garantiza que los datos no sean alterados durante la transmisión.
- ▶ **Autenticación:** Verifica la identidad del servidor para evitar ataques de suplantación.

HTTP: SSL/TLS y seguridad en la comunicación (Cont.)

Uso de HTTPS:

- ▶ HTTPS (HTTP + TLS) proporciona una **capa adicional de seguridad**.
- ▶ Es **obligatorio** en aplicaciones modernas y recomendado por motores de búsqueda como Google.
- ▶ **Requiere un certificado** SSL/TLS emitido por una autoridad de certificación (CA).

HTTP: Estructura

El formato tanto del mensaje como de la respuesta es el siguiente:

```
<Línea inicial>  
Encabezado-1: valor-1  
...  
Encabezado-n: valor-n  
<Cuerpo del mensaje (Opcional)>
```

La línea inicial es diferente en las solicitudes y en las respuestas:

- ▶ En las solicitudes está formada por tres campos que se separan con un espacio en blanco: “<Método> <Recurso> <Versión del Protocolo>”. Por ejemplo, "GET / HTTP/2"
- ▶ La línea inicial de una respuesta tiene tres campos separados por un espacio: “<Versión del Protocolo> <Código de Estado> <Mensaje de Estado>”. Por ejemplo, "HTTP/2 200 OK"

HTTP: Estructura (Cont.)

Ejemplo de solicitud HTTP

```
GET / HTTP/2  
Host: www.ubp.edu.ar  
User-Agent: Mozilla/5.0  
Accept: text/html
```

Ejemplo de respuesta HTTP

```
HTTP/2 200 OK  
Content-Type: text/html  
Content-Length: 512  
Server: Apache
```

HTTP: Métodos

- ▶ **GET:** Utilizado para **solicitar un documento o recurso específico**. Es la **operación por defecto** al navegar por la Web. Debe tenerse en cuenta que cuando se utiliza el método con propósitos de transferir datos, su uso es poco conveniente ya que el URL tiene una **limitación de 8.192 caracteres** y muchas veces podría truncarse los datos que se envían al servidor aun codificados.
- ▶ **POST:** Empleado para **transferir datos** desde el cliente al servidor, los cuales serán procesados. El ejemplo clásico es el envío de datos de un formulario.
- ▶ **PUT:** Utilizado para **almacenar o actualizar recursos** en el servidor, por ejemplo, archivos.

HTTP: Métodos (Cont.)

- ▶ **DELETE**: Empleado para **borrar recursos** del servidor.
- ▶ **HEAD**: Es una operación especial que tan sólo nos **recupera información del recurso**, como el tamaño, la fecha de modificación, tipo, etc. Lo suelen utilizar los navegadores o servidores Proxy para comprobar el estado de su caché u otras operaciones. Es similar a la operación GET pero sin el cuerpo de la respuesta.
- ▶ **TRACE**: Solicita que el cuerpo del mensaje sea retornado como se lo envió, es empleado principalmente para **depuración o debugging**.
- ▶ **OPTIONS**: Proporciona una forma para que el cliente **consulte al servidor sobre sus capacidades o de un recurso específico**.

HTTP: Métodos (Cont.)

- ▶ **CONNECT:** Se utiliza para **saber si se tiene acceso** a un servidor, no necesariamente la petición llega al servidor. Este método se utiliza principalmente para saber si un proxy nos da acceso a un host bajo condiciones especiales.

HTTP: Códigos de estado de respuesta

Código	Significado
100 - 199	Indica un mensaje informativo
200 - 299	Indica éxito de alguna clase
300 - 399	Redirecciona el cliente a otra URL
400 - 499	Indica un error del lado del cliente
500 - 599	Indica un error del lado del servidor

HTTP: Ejemplos de códigos de estados

- ▶ **200 OK**
Pedido exitoso, objeto pedido contenido en el cuerpo.
- ▶ **301 Moved Permanently**
Objeto pedido ha sido movido a una nueva ubicación especificada en el mensaje respuesta.
- ▶ **400 Bad request**
Mensaje pedido no comprendido por el servidor.
- ▶ **404 Not Found**
Documento pedido no encontrado en el servidor.
- ▶ **505 HTTP Version Not Supported**
Versión del protocolo HTTP no soportada por el servidor.

MIME

Los tipos MIME (Multipurpose Internet Mail Extensions) son un estándar que **permite identificar el tipo de contenido que se transfiere** a través de protocolos como HTTP.

Indican al navegador **cómo debe interpretar el recurso recibido**.

Importancia:

- ▶ Permiten la **correcta interpretación de archivos** por parte de navegadores y servidores.
- ▶ Facilitan la **transmisión de distintos tipos de contenido** (imágenes, video, audio, archivos, etc.) a través de HTTP.
- ▶ Se utilizan en el encabezado **Content-Type** de las respuestas HTTP.

MIME: Estructura

Un tipo MIME está compuesto por dos partes separadas por una barra /:

- ▶ **Grupo:** Define la categoría del contenido (ejemplo: text, image, video).
- ▶ **Subtipo:** Especifica el formato exacto del contenido.

Ejemplos:

- ▶ **text/html:** Define documentos HTML.
- ▶ **image/png:** Define imágenes en formato PNG.
- ▶ **video/mp4:** Define videos en formato MP4.

Uso de comodines (*): Se puede utilizar * para referirse a todos los subtipos dentro de un grupo. Por ejemplo, image/* abarca cualquier tipo de imagen (JPG, PNG, GIF, etc.).

